CXRF Memory Map

`v0.10.0` Commander X16 Runtime Framework — the live ledger of contended address space

Every byte of contended space is accounted for here. The prose ledger is `docs/memory-map.md`; the assembler-enforced source of truth is `kernel/resident/banks.inc`, `kernel/resident/zp.inc` and `kernel/kernel.cfg`. Budget figures are the v0.10.0 build, read back by `tools/mapreport.py`.

hardware / fixed · KERNAL / system · x16lib · CXRF kernel · application

---

Code budget `mapreport.py · v0.10.0`

Every code region is a hard ceiling — ld65 fails the link on overflow. Green is comfortable, amber is worth a look (≥60%), red is tight (≥85%). The resident image and the overlay window are the walls to watch.

RESIDENT $81A9–$95FF	5013 / 5207
OVL window $9600–$9EFF	2004 / 2304
Jump table $8010–$81A4	105 / 135 sl
BANK 17 gfx extras	6286 / 8192
BANK 19 extra shapes	4818 / 8192
BANK 16 widgets	4654 / 8192
BANK 5 dialogs	3478 / 8192
BANK 18 fs/aud/spr	2238 / 8192
BANK 3 mode-0 img	2004 / 8192
BANK 2 UI core	1768 / 8192
BANK 4 mode-1 img	1543 / 8192

Zero page $00–$FF

RANGE	OWNER	NOTES
$00–$01	hardware	`RAM_BANK` / `ROM_BANK` registers
$02–$21	KERNAL r0–r15	caller-save scratch; never live across a CXRF API call; IRQ callbacks bracket with `irq_save_regs`
$22–$31	x16lib	`X16_P0..P7` + `X16_T0..T7`, fixed as vendored
$32–$5F	CXRF kernel	46-byte window (`zp.inc`). Used $32–$54 (font, event, far-call, menu, dir, clipboard, joystick, icon, vram-streamer); free $55–$5F
$60–$7F	application	guaranteed untouched by kernel and IRQ
$80–$FF	KERNAL/BASIC/DOS	never touch

Low RAM & the kernel image `$0100–$9FFF`

RANGE	SIZE	CONTENTS
`$0100–$01FF`	256	CPU stack — kernel ≤48 B below caller SP per API call, ≤16 in IRQ
`$0200–$07FF`	1,536	KERNAL/DOS — untouched (stock R49 IRQ handler is at $038B; we never go near it). Loader stages CXAP headers at $0400–$041F
`$0801–$7FFF`	~30 KB	application — loaded/reset by the kernel loader; the loader's hard ceiling is $7FFF
`$8000–$800F`	16	ABI header — magic `CXOS`, ABI version word, slot count, init vector, `cx_hdr_shell` / `CX_SHELL_SEL` (survive app loads)
`$8010–$81A4`	405	jump table — 3-byte JMP slots, append-only, slot *n* at $8010+n·3 forever. **105 slots used** ($8010–$814A); reserve caps at 135 (30 free)
`$81A5–$81A8`	4	build word `CX_KBUILD` ($0602) — stage-0 checks it against the banked files
`$81A9–$95FF`	5,207	resident kernel — event core + IRQ, font hot path, region routing, far-call trampoline, loader, clipboard byte-mover, port manager. **~194 B free (96% full)**
`$9600–$9EFF`	2,304	graphics port (OVL) — the current engine image, copied from its bank by `cx_gfx_mode`; or the tile-text port (`OV3T`) swapped in by `cx_tile_text`
`$9F00–$9FFF`	256	I/O — VERA, VIA, PSG-through-VERA, etc. Past here is not memory

Banked RAM `$A000–$BFFF window · bank register $00` `8 KB / bank`

The kernel's code banks are two files: `CXBANKS.BIN` (banks 2–5) and `CXBANKS2.BIN` (banks 16–19), 32 KB each. Data banks sit between; apps sit above `CX_APP_BANK_FLOOR = 20`. Used/free are the v0.10.0 build.

BANK	THEME	SEGMENT / OWNER	USED	FREE
0	KERNAL	reserved (KERNAL's)	–	–
1	kernel data	system font ($A000), desktop state, theme, font metrics, event overflow	–	–
2	**UI core**	`B2CODE`: menus + theme + DA manager + the `b2_table` local jump table	1,768	6,424
3	mode-0 image	`OV0CODE` (2bpp GUI engine), copied to the OVL window to run	2,004	6,188
4	mode-1 image	`OV1CODE` (8bpp bitmap engine)	1,543	6,649
5	**dialogs**	`B5CODE`: dialog/alert/prompt/panel + the mode-2/3 & tile-text port images (`OV2/OV3/OV3TCODE`)	3,478	4,714
6–8	glyph cache	pre-shifted 2bpp cache (95 glyphs × 192 B, 42/bank) + CXF sources	–	–
9	desk accessory	the open `.CXD` (`cx_da_open` loads it at $A000)	–	–
10–13	clipboard	typed text / bitmap-rect, up to 32 KB	–	–
14–15	save-under	dialog save-unders / DA saved state	–	–
16	**widgets**	`B16CODE`: the whole toolkit (code + `wg_*` state) + `wg_paint_t`; icon & `WG_HIT` widgets	4,654	3,538
17	**graphics extras**	`B17CODE`: base shapes (circle/disc/ellipse/flood) + tile machinery + dirty rects + icon sheet + palette API **(tightest: 77%)**	6,286	1,906

BANK	THEME	SEGMENT / OWNER	USED	FREE
18	fs / sys / audio / sprites	`B18CODE` : dir walk + `cx_file_load` + `cx_vload` / `cx_bload` + DOS channel + font cold half; PSG + YM audio + sprites; clipboard put/get/type orchestration	2,238	5,954
19	extra shapes	`B19CODE` : the dispatched `cx_gfx_shape` family — polygon / fpolygon / arc / pie + the sin/cos table	4,818	3,374
20+	the app's	`cx_bload` targets (refuses < 20), window backing store, allocations, file buffers. 44 app banks on 512 KB, 236 on 2 MB	–	–

Each of banks 16–19 opens with an 8-byte signature (`"CXB"` , bank #, `CX_KBUILD` , code size) at $A000; stage-0 verifies it, so a hand-copied SD card carrying one stale file refuses at boot instead of crashing in a far call.

ROM `$C000–$FFFF window · bank register $01` `16 KB / bank`

Banks 0–31 are the stock R49 system ROM — CXRF adds nothing to them (running on stock ROM is the whole premise). The only ROM CXRF emits is the optional cartridge (`build.ps1 -Cart`): five 16 KB banks loaded at bank 32.

ROM BANK(S)	OWNER	CONTENTS
0–31	stock KERNAL ROM	KERNAL / BASIC / CBDOS / ... — untouched; CXRF requires the R49 image
32	CXRF cartridge	`"CX16"` signature at $C000, boot stub ($C004), the relocated low-RAM copier, then the resident image + system font as data
33–34	CXRF cartridge	`CXBANKS.BIN` (32 KB) → copied to RAM banks 2–5 at boot
35–36	CXRF cartridge	`CXBANKS2.BIN` (32 KB) → copied to RAM banks 16–19 at boot
37	CXRF cartridge `-D CART_APP`	standalone build only: one app's `.CXA` baked at $C000, copied to $0801 and run

The KERNAL scans bank 32 $C000 for `"CX16"` and jumps to $C004, so the cart needs no `AUTOBOOT.X16` and no ROM patch. Banks 32–36 are the default five; bank 37 exists only in the `CART_APP` standalone build.

VRAM `128 KB · $00000–$1FFFF` `reinterpreted per mode`

The contended graphics resource. The layout below is **mode 0 (desktop)** / **mode 1 (bitmap)**; the hardware regions at the top ($1F9C0+) are fixed and shared by every mode.

RANGE	SIZE	CONTENTS (MODE 0 / 1)
$00000–$12BFF	76,800	framebuffer 640×480 @2bpp, 160 B/row
$12C00–$12FFF	1,024	scratch strip (small save-unders, blit staging)
$13000–$130FF	256	KERNAL mouse pointer image (r49 `sprite_addr`)
$13100–$170FF	16,384	menu / drop-down save-under strips (`fx_copy` restore)
$17100–$1DFFF	28,416	icon / pattern sheets, extra save-under
$1E000–$1EFFF	4,096	CXRF sprite images (extra cursors, drag outlines)
$1F000–$1F7FF	2,048	KERNAL charset — kept for the panic/debug console
$1F800–$1F9BF	448	x16lib `VRAM_FX_SCRATCH` = $1F800

RANGE	SIZE	CONTENTS (MODE 0 / 1)
$1F9C0–$1F9FF	64	PSG registers (hardware)
$1FA00–$1FBFF	512	palette (hardware)
$1FC00–$1FFFF	1,024	sprite attributes (hardware)

RANGE	CONTENTS (MODE 2 · TILE)
$00000–$0FFFF	tileset — up to 64 KB, 1,024 tiles at 8bpp
$10000	layer-0 game map (64×32 × 2 B = 4 KB); mapbase constant at every depth
$11000	layer-1 game map
$12000	text-overlay map (`cx_tile_text` — menus/widgets/dialogs)
$14000	layer-0 shadow map (`cx_tile_dbuf` / `flip` double-buffer)
$15000	layer-1 shadow map
$1F000	KERNAL charset staged by `ov2_init` (tile-text glyphs)

Sprite images & save-under strips are unused in tile mode — the game owns that space. The $1F9C0+ hardware regions are shared with mode 0/1.

CXRF v0.10.0 · generated from `docs/memory-map.md` and the kernel build (`tools/mapreport.py`). Change this map in the same commit as the code that claims or releases a region. Budget figures shift every build — run `python tools/mapreport.py` for today's.